第五章 系统测试与实验分析

前两章分别完成了系统总体设计和板端实现说明。本章在此基础上对系统进行实验测试与结果分析，重点验证三类问题：第一，训练得到的单类无人机检测模型是否能够在公开无人机视频中产生有效检测结果；第二，模型转换为 RKNN 后能否在 RK3588 开发板上稳定运行；第三，不同实时检测策略和多 context NPU 并行方案对系统吞吐、延迟和显示稳定性的影响。

由于无人机真实飞行测试场景受场地、天气和安全条件限制，本文采用“公开无人机视频固定输入 + RK3588 板端实时 RTSP 验证”的实验路线。固定视频实验用于保证输入可重复，便于比较不同配置；实时 RTSP 实验用于验证系统在实际部署路径中的端到端表现。这样可以在缺少长期真实飞行场景的情况下，仍然形成较完整的模型有效性、板端可运行性和系统优化效果证据。

5.1 实验环境与评价指标

本文实验主要包括 PC/WSL 端模型训练与评估、RK3588 板端固定视频验证以及 RK3588 实时 RTSP 推流验证。PC/WSL 端用于训练 YOLOv10 单类无人机模型，并完成 PyTorch、ONNX 与 RKNN 文件的生成；RK3588 板端用于加载 RKNN 模型并执行视频检测、结果绘制和 RTSP 输出。

表 5.1 给出了本章实验涉及的主要环境和输入条件。

表 5.1 实验环境与输入条件

类别	内容	说明
训练主机	Windows 与 WSL 环境	用于 YOLOv10 模型训练、ONNX 导出和 RKNN 转换准备
部署平台	RK3588 开发板	用于加载 RKNN 模型并调用 NPU 推理
部署程序	rk_yolo_video、rk_yolo_live_rtsp	分别用于固定视频检测和实时 RTSP 检测
主要模型	FP RKNN 模型	当前稳定的无人机单类 RKNN 模型
输入视频	公开视频样本	用于短视频验证和较长序列验证
输出内容	带框视频、CSV、ROI JSONL、RTSP 流、运行日志	用于定性观察和定量分析

评价指标分为模型检测指标和系统运行指标两类。模型检测指标主要包括 precision、recall、mAP50 和 mAP50-95，用于反映模型在验证集上的检测效果。系统运行指标主要包括 stream_fps、npu_fps、端到端延迟、CPU 占用、内存占用和丢帧数量，用于评价实时检测系统在 RK3588 上的运行表现。

表 5.2 给出了本章使用的主要系统指标含义。

表 5.2 系统运行指标说明

指标	含义	分析作用
stream_fps	实际输出视频流帧率	反映观看端画面流畅程度
npu_fps	实际执行 NPU 推理的频率	反映模型检测刷新强度
端到端延迟	从采集到输出的整体延迟	反映实时响应能力
work_ms	单帧主要处理耗时	用于分析处理链路负载
CPU 占用	程序运行时 CPU 使用比例	反映板端资源压力
RSS 内存	程序常驻内存占用	反映多 context 和缓存开销
dropped_capture / dropped_publish	采集或发布阶段丢帧数量	反映系统是否出现队列积压

需要说明的是，本文当前阶段的重点是嵌入式部署与 NPU 多线程优化，而不是提出新的检测网络结构。因此，本章在讨论模型效果时更强调“模型能否作为系统输入稳定工作”，在讨论系统实验时则更强调“RK3588 上不同运行策略的吞吐与延迟差异”。

5.2 无人机检测模型训练结果

本文使用 YOLOv10n 作为基础模型，并在单类无人机数据集上进行迁移训练。训练轮数为 50 epoch，输入尺寸为 640，batch size 为 16，训练完成后得到 PyTorch 权重文件 best.pt。该模型随后被导出为 ONNX，并进一步转换为 RK3588 可加载的 RKNN 模型。

表 5.3 给出了训练完成后验证集上的主要指标。

表 5.3 单类无人机模型训练结果

指标	数值
Precision	0.8291
Recall	0.8426
mAP50	0.8901
mAP50-95	0.5095
训练轮数	50
输入尺寸	640 × 640

从表 5.3 可以看出，当前模型在验证集上具有较高的 mAP50，说明其能够较好地区分无人机目标与背景。mAP50-95 相对低于 mAP50，说明在更严格的定位阈值下，模型的边界框精度仍有提升空间。这与小目标无人机检测任务特点一致：远距离无人机在图像中尺寸较小，轻微的边界框偏移就会对高 IoU 阈值下的评价结果产生较大影响。

在论文表述中，本文不将该训练结果描述为算法创新，而是将其作为后续 RK3588 部署和实时检测实验的模型基础。这样可以避免夸大模型本身贡献，同时突出本课题的系统移植和硬件优化重点。

5.3 公开视频固定输入验证

为了弥补真实无人机飞行场景不易获得的问题，本文引入公开无人机视频作为固定输入测试源。公开视频层的作用不是直接替代完整数据集训练，而是为系统提供可重复的视频输入，使不同程序、不同模型和不同运行策略可以在相同场景下进行比较。

PC/WSL 端首先使用 best.pt 对 anti_uav_fig1.mp4 进行离线检测。该视频共 130 帧，在置信度阈值 0.35 下，检测结果为：16 帧存在检测框，总检测数为 18，最高置信度为 0.6216。这说明当前模型能够在新的公开无人机视频源中产生目标响应。

随后，本文将模型部署到 RK3588 板端进行固定视频验证。早期直接转换的无人机 RKNN 模型曾在 rknn_run 阶段出现运行崩溃。后续通过重新导出 end2end=False 的 ONNX 模型，并在 C++ 后处理模块中适配单类输出头 [1, 5, 8400]，恢复了板端运行路径。该过程说明模型部署并不是单纯格式转换，还需要与输出张量结构和后处理逻辑保持一致。

表 5.4 给出了 RK3588 固定视频验证结果。

表 5.4 RK3588 固定视频检测结果

视频样本	帧数	有检测帧	检测数	平均推理
短公开视频	130	30	30	109.43 ms
DUT video01	1050	846	846	67.61 ms

从表 5.4 可以看出，恢复后的无人机 RKNN 模型不仅能够在短视频上运行，也能够在较长的 DUT-Anti-UAV 序列上稳定产生检测结果。其中 video01.mp4 共 1050 帧，有 846 帧输出检测框，说明该视频适合作为后续重复实验和论文定性展示的主要公开视频样本。

固定视频验证的意义在于，它将模型可用性、RKNN 运行稳定性和输出记录能力统一到同一条可重复路径中。相比只进行实时摄像头演示，固定视频可以保存带框视频、逐帧 CSV 和 ROI JSONL 文件，更适合论文实验记录和后续误差分析。

5.4 实时 RTSP 首轮配置对比

在固定视频路径跑通后，本文进一步测试实时 RTSP 检测程序。首轮实时实验比较了基线全帧检测、多 context 全帧检测和策略优化检测三类配置。其目标不是单纯追求最高帧率，而是观察不同配置在吞吐和延迟之间的差异。

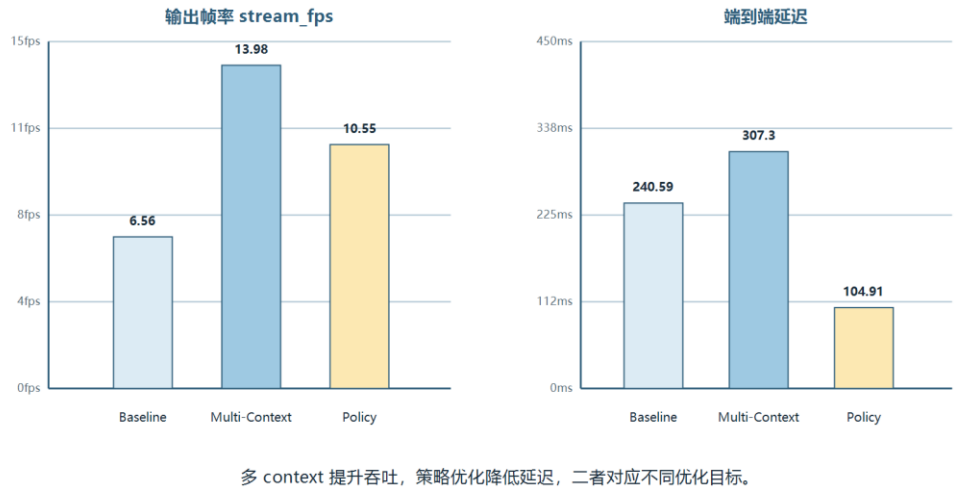

图 5.1 实时 RTSP 首轮配置对比

表 5.5 给出了首轮实时 RTSP 实验的主要结果。

表 5.5 实时 RTSP 首轮配置实验结果

配置	stream	npu	工作耗时	端到端延迟	采集丢帧	说明
Baseline	6.56	6.56	152.49 ms	240.59 ms	80	单 worker 全帧推理基线
Multi-Context	13.98	13.98	180.16 ms	307.30 ms	0	双 context 全帧推理，吞吐最高
Policy-Optimized	10.55	5.04	0.00 ms	104.91 ms	1	跳帧与复用策略，延迟最低

表中 Policy-Optimized 的平均推理/工作耗时显示为 0.00 ms，是因为采样日志对应的是复用帧，而不是完整 NPU 推理帧。因此该数值不能直接与全帧推理配置比较，但其端到端延迟仍然反映了策略模式下的实时观看响应。

从实验结果可以看出，Multi-Context 配置显著提高了输出帧率和 NPU 推理频率，但端到端延迟也随之升高。Policy-Optimized 配置没有追求每帧都执行完整检测，而是通过跳帧检测、动态 ROI 和轻量跟踪保持显示连续性，端到端延迟最低。该结果

说明实时系统优化不能只看 NPU 吞吐，还需要综合考虑检测刷新频率、队列积压、输出延迟和观看体验。

5.5 检测间隔与框平滑策略实验

在实时观看过程中，检测框曾出现左右抖动现象。分析认为，这与检测间隔较大、轻量跟踪漂移以及框平滑未启用有关。为此，本文比较了 detect_every_n 和 box smoothing 对实时显示稳定性的影响。

首先，在无目标或弱目标实时场景下比较了 detect_every_n=2、3、4 三种策略。实验结果如表 5.6 所示。

表 5.6 检测间隔策略实验结果

配置	stream	npu	延迟	CPU	RSS	说明
N=2	10.02	6.68	175.20 ms	43.0%	132.37 MB	NPU 刷新强度较高，但延迟较高
N=3	10.91	4.96	107.18 ms	37.1%	127.36 MB	帧率、延迟和资源占用较均衡
N=4	9.54	3.82	95.54 ms	33.5%	127.97 MB	延迟较低，但刷新强度下降

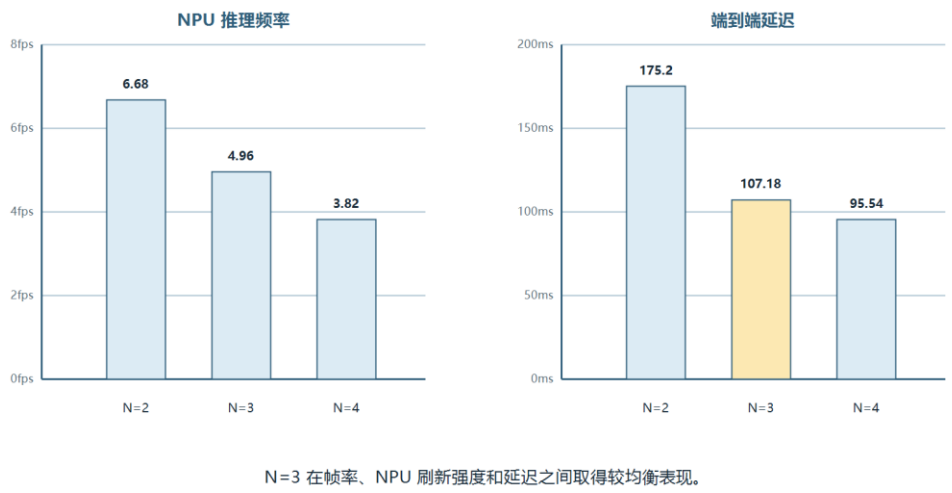

图 5.2 检测间隔策略性能变化

从表 5.6 可以看出，随着 detect_every_n 增大，NPU 推理频率降低，端到端延迟总体下降，但目标刷新能力也随之降低。N=4 虽然延迟较低，但输出帧率和检测

刷新强度均下降；N=2 虽然检测更频繁，但延迟明显较高。因此，在该轮实验中 N=3 可作为无目标实时场景下的较优折中点。

后续在公开视频 video01.mp4 的目标场景中再次比较 N=3 和 N=2，并加入框平滑。结果表明，N=2 配合 box_smooth=on 后可以明显减轻检测框左右抖动，虽然 NPU 调用频率提高，但输出帧率仍保持在约 10 FPS。表 5.7 给出了该轮消融实验结果。

表 5.7 跳帧检测与框平滑消融实验结果

配置	stream	npu	延迟	工作耗时	观察结果
N=3，平滑关闭	9.95	3.32	102.60 ms	101.99 ms	框抖动较明显
N=3，平滑开启	9.93	3.31	108.05 ms	107.03 ms	抖动减轻，但刷新频率不变
N=2，平滑开启	10.36	5.18	111.41 ms	110.76 ms	检测框更稳定，实时性仍可接受

综合两轮实验可知，若系统处于纯实时观看、目标不明显或演示场景较简单时，detect_every_n=3 可以作为低延迟默认配置；若视频中存在无人机目标且需要更稳定的框位置，detect_every_n=2 加框平滑更适合作为当前演示配置。

5.6 多 context NPU 并行推理实验

导师提出的主要优化方向是利用 RK3588 NPU 做多线程并行以提高效率。为验证这一方向，本文在实时 RTSP 程序中加入多 context 推理机制，即创建多个 RKNN 检测器实例，每个 worker 拥有独立 RKNN context，并由调度队列分配待检测帧。

表 5.8 给出了多 context NPU 并行实验结果。

表 5.8 多 context NPU 并行推理实验结果

配置	N	W	stream	npu	延迟	CPU	RSS	丢帧
A	2	1	10.37	5.18	109.06 ms	121.5%	205.5 MB	0 capture / 0 publish
B	1	1	8.59	8.59	268.89 ms	144.7%	206.3 MB	116 capture / 0 publish
C	1	2	10.63	10.64	173.58 ms	143.4%	233.7 MB	0 capture / 0 publish
D	1	3	10.45	10.45	179.59 ms	147.7%	239.0 MB	0 capture / 0 publish

实验结果表明，单 context 每帧检测并不适合作为实时默认配置。虽然其 NPU 推理频率高于跳帧策略，但 stream_fps 下降到 8.59 FPS，端到端延迟升高到 268.89 ms，并出现明显采集丢帧。双 context 每帧检测将 NPU FPS 提升到 10.64 FPS，同时消除了采集丢帧，说明多 context 机制确实提高了每帧检测场景下的吞吐能力。

然而，三 context 并没有进一步提升性能，反而在延迟和内存占用方面略差于双 context。这说明在当前系统中，两个 context 之后瓶颈已经不再只是 NPU context 数量，输入设置、内存传输、线程调度和输出发布等环节开始限制整体性能。

因此，多 context 实验可形成两条结论。第一，从硬件优化角度看，双 context 是本实验配置下较有效的 NPU 并行配置，实验结果表明 RK3588 NPU 多 context 并行对吞吐有实际提升。第二，从实时演示角度看，detect_every_n=2 或 detect_every_n=3 的策略模式仍然更适合作为低延迟观看配置。

5.7 阶段耗时与瓶颈分析

为了进一步解释为什么三 context 收益有限，本文加入 RK_YOLO_PROFILE 开关，对推理阶段进行细粒度计时。该开关可以记录 prepare、rknn_inputs_set、rknn_run、outputs_get、decode 和 render 等阶段耗时。

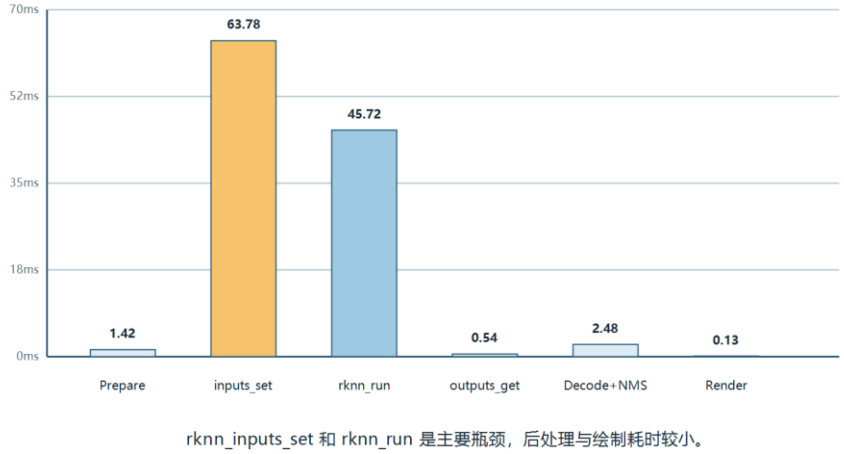

图 5.3 RKNN 推理阶段耗时分布

表 5.9 给出了单 context 和双 context 条件下的阶段耗时对比。

表 5.9 RKNN 推理阶段耗时分析

阶段	N=1，W=1	N=1，W=2
Stream FPS	8.57	10.53
NPU FPS	8.57	10.53
端到端延迟	255.52 ms	179.52 ms
总推理工作耗时	114.48 ms	113.96 ms
Prepare	3.62 ms	1.42 ms
rknn_inputs_set	62.01 ms	63.78 ms
rknn_run	45.82 ms	45.72 ms
rknn_outputs_get	0.49 ms	0.54 ms
Decode + NMS	2.40 ms	2.48 ms
Render	0.15 ms	0.13 ms

从表 5.9 可以看出，双 context 并没有降低单次推理本身的总工作耗时，两个配置下总推理耗时都约为 114 ms。双 context 的提升来自不同推理任务之间的重叠执行，而不是单帧推理速度变快。

更重要的是，rknn_inputs_set 的耗时约为 62 至 64 ms，甚至高于 rknn_run 的约 46 ms。这表明当前瓶颈并不在后处理或绘制阶段。Decode + NMS 仅约 2.4 ms，

Render 仅约 0.1 至 0.2 ms，继续优化画框或标签绘制对整体吞吐帮助有限。后续若要进一步提升性能，应优先考虑输入数据传输、预处理和内存布局优化。

5.8 Zero-copy 输入路径探索

基于阶段耗时分析，本文进一步尝试 RKNN zero-copy 输入路径，希望降低 rknn_inputs_set 的输入设置开销。实验通过 RK_YOLO_ZERO_COPY_INPUT 环境变量控制，默认关闭，以避免影响稳定演示路径。

在 zero-copy 实验中，程序使用 rknn_create_mem 创建 RKNN 管理的输入内存，并通过 rknn_set_io_mem 将其绑定到模型输入。实验结果如表 5.10 所示。

表 5.10 Zero-copy 输入实验结果

指标	标准路径	Zero-copy
Stream FPS	8.57	8.57
NPU FPS	8.57	8.57
端到端延迟	255.52 ms	267.19 ms
总推理工作耗时	114.48 ms	118.93 ms
输入设置或更新	62.01 ms	0.43 ms
rknn_run	45.82 ms	111.36 ms
Decode + NMS	2.40 ms	2.64 ms

从表 5.10 可以看出，zero-copy 路径确实将显式输入更新耗时从约 62 ms 降低到 1 ms 以下，但 rknn_run 的耗时从约 46 ms 上升到约 111 ms，最终总耗时并未下降。这说明输入转换或同步开销并没有真正消失，而是从 rknn_inputs_set 阶段转移到了 rknn_run 阶段。

造成这一现象的可能原因是当前模型输入张量与应用侧准备的数据类型并不完全匹配。例如应用侧仍以 uint8 图像作为输入，而模型或 RKNN Runtime 内部仍需要执行格式或类型转换。因此，简单绑定 RKNN 输入内存并不足以得到真正的零拷贝加速。

该实验虽然没有带来性能提升，但对论文仍然有价值。它说明本文的优化过程不是只停留在经验调参，而是通过阶段计时定位瓶颈，并对输入内存路径进行验证。当前稳定演示路径应继续保持 RK_YOLO_ZERO_COPY_INPUT=0，zero-copy 与 RGA 硬件预处理可作为后续优化方向。

5.9 综合讨论与推荐配置

综合本章实验，可以得到以下结论。

第一，当前单类无人机模型已经能够作为 RK3588 检测系统的有效输入。训练结果显示 mAP50 达到 0.8901，公开无人机视频验证也表明模型可以在新场景中产生检测框。板端固定视频实验进一步表明，恢复后的 end2end=False RKNN 模型可以在 RK3588 上稳定运行。

第二，固定视频和实时 RTSP 两条路径具有不同实验价值。固定视频路径可重复性强，适合模型可用性验证、输出视频生成和 CSV 结果分析；实时 RTSP 路径更接近实际部署，适合评价端到端延迟、观看流畅性和系统稳定性。

第三，RK3588 NPU 多 context 并行对每帧检测吞吐有明显帮助。相比单 context 每帧检测，双 context 将 NPU FPS 从 8.59 提升到 10.64，并消除了采集丢帧。但三 context 没有继续带来收益，说明系统已出现输入传输和调度瓶颈。

第四，实际演示配置不应只追求最高 NPU FPS。多 context 全帧检测吞吐更高，但端到端延迟较大；策略优化模式通过跳帧检测、动态 ROI 和框平滑获得更好的观看响应。因此，系统级推荐配置和硬件优化实验配置应分开表述。

表 5.11 给出了当前阶段推荐配置。

表 5.11 当前推荐配置

使用目标	推荐配置	理由
实时观看演示	detect_every_n=2，infer_workers=1，box_smooth=on，dynamic_roi=on	检测框更稳定，输出约 10 FPS，延迟约 110 ms
低延迟无目标巡检	detect_every_n=3，infer_workers=1，dynamic_roi=on	延迟较低，资源占用较小，适合一般实时查看
NPU 多线程实验	detect_every_n=1，infer_workers=2	能体现双 context 对吞吐的提升，适合作为硬件优化论证
后续优化方向	输入内存路径、RGA 预处理、量化输入匹配	当前瓶颈主要在 rknn_inputs_set 和 rknn_run

需要特别说明的是，INT8 量化和 RGA 硬件预处理目前不应写成已经完成的稳定优化结果。根据当前实验，FP RKNN 模型是稳定主路径，zero-copy 输入路径仍处于探索状态。论文中应将这些内容表述为后续优化方向，而不是已验证成果。

5.10 本章小结

本章围绕 RK3588 无人机检测系统进行了模型、固定视频、实时 RTSP、多 context 并行和阶段耗时等实验分析。训练结果表明当前单类无人机模型具备基本检测能力；公开视频固定输入验证证明恢复后的无人机 RKNN 模型能够在 RK3588 上

稳定运行；实时 RTSP 实验说明多 context 可以提升吞吐，而检测间隔、动态 ROI 和框平滑更适合改善低延迟观看体验。

阶段耗时分析进一步指出，当前系统的主要瓶颈不是后处理和画框，而是 RKNN 输入设置和 NPU 执行阶段。zero-copy 实验表明，简单绑定输入内存不能直接解决性能问题，后续需要结合输入数据类型、模型量化方式和硬件预处理路径继续优化。

总体来看，本文已经形成了较完整的实验链路：模型训练、RKNN 转换、板端固定视频验证、实时 RTSP 演示、多 context 并行对比和瓶颈分析。下一章将基于前文设计、实现和实验结果，对全文工作进行总结，并讨论后续可改进方向。